

Exploratory Data Analysis of Retail Sales and Customer Behaviour Using Python

❖ **Objective:**

- To explore and analyze retail sales data to identify trends, customer purchasing patterns and product performance, enabling better business insights and decision-making.

❖ **Dataset Overview:**

- Total Records (Rows): 60,398
- Total Fields (Columns): 28
- Key Fields:
 - Order ID, Order Date, Ship Date
 - Customer Name, Customer ID, Country
 - Product Name, Category, Sub-Category
 - Sales, Quantity, Discount, Profit

❖ **Tools & Technologies:**

- Programming Language: Python
- Libraries Used: Pandas, NumPy
- Environment: Google Colab
- Dataset Format: CSV(Kaggle)

❖ **Steps Performed:**

- 1) Data Loading & Copying – Imported dataset and created a working copy:
 - `(df = full_data_revenue.copy())`
- 2) Data Overview – Used `df.info()`, `df.describe()`, and `df.isnull().sum()` to understand structure, summary stats and missing values.
- 3) 40 Unique Analysis Queries – Covered multiple aspects of sales & customer behavior.
- 4) Categorized Queries:
 - Basic Descriptive Analysis.
 - Time-based Trends.
 - Product & Category Insights.
 - Customer-based Analysis.
 - Advanced / RFM-style Metrics.
 - Additional Insights & Patterns.

❖ Python Code Snapshots:

➤ Data loading:

```
import kagglehub
import pandas as pd

# Download latest version
path = kagglehub.dataset_download("rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl")

print("Path to dataset files:", path)
```

Downloading from https://www.kaggle.com/api/v1/datasets/download/rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl?dataset_version_number=2...
100%[██████████] 1.19M/1.19M [00:00<00:00, 55.7MB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl/versions/2

```
[ ] xls /root/.cache/kagglehub/datasets/rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl/versions/2
```

Customer_report_cleaned_data.csv g_fact_sales.csv
g_dim_customers.csv Product_report_cleaned_data.csv
g_dim_products.csv

```
[ ] fact_sales = pd.read_csv(r'/root/.cache/kagglehub/datasets/rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl/versions/2/g_fact_sales.csv')
```

```
[ ] dim_products = pd.read_csv(r'/root/.cache/kagglehub/datasets/rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl/versions/2/g_dim_products.csv')
```

```
[ ] dim_customers = pd.read_csv(r'/root/.cache/kagglehub/datasets/rizwanbinakbar/cleaned-retail-customer-dataset-sql-based-etl/versions/2/g_dim_customers.csv', encoding='latin1')
```

fact_sales.head()

	order_number	customer_key	product_key	order_date	shipping_date	due_date	sales	quantity	price
0	SO43697	10769	20	2010-12-29	2011-01-05	2011-01-10	3578	1	3578
1	SO43698	17390	9	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
2	SO43699	14864	9	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
3	SO43700	3502	41	2010-12-29	2011-01-05	2011-01-10	699	1	699
4	SO43701	4	9	2010-12-29	2011-01-05	2011-01-10	3400	1	3400

Next steps: [View recommended plots](#) [New interactive sheet](#)

```
[7] dim_products.head()
```

	product_key	product_id	product_number	product_name	category_id	category	sub_category	maintenance	product_line	cost	start_date
0	1	210	FR-R92B-58	HL Road Frame - Black- 58	CO_RF	Components	Road Frames	Yes/r	Road	0	2003-07-01
1	2	211	FR-R92R-58	HL Road Frame - Red- 58	CO_RF	Components	Road Frames	Yes/r	Road	0	2003-07-01
2	3	348	BK-M82B-38	Mountain-100 Black- 38	BI_MB	Bikes	Mountain Bikes	Yes/r	Mountain	1898	2011-07-01
3	4	349	BK-M82B-42	Mountain-100 Black- 42	BI_MB	Bikes	Mountain Bikes	Yes/r	Mountain	1898	2011-07-01
4	5	350	BK-M82B-44	Mountain-100 Black- 44	BI_MB	Bikes	Mountain Bikes	Yes/r	Mountain	1898	2011-07-01

Next steps: [View recommended plots](#) [New interactive sheet](#)

```
[8] dim_customers.head()
```

	customer_key	customer_id	customer_number	first_name	last_name	country	gender	marital_status	birth_date	create_date
0	1	11000	AW00011000	Jon	Yang	Australia	Male	Married	1971-10-06	10/6/2025/r
1	2	11001	AW00011001	Eugene	Huang	Australia	Male	Single	1976-05-10	10/6/2025/r
2	3	11002	AW00011002	Ruben	Tomes	Australia	Male	Married	1971-02-09	10/6/2025/r
3	4	11003	AW00011003	Christy	Zhu	Australia	Female	Single	1973-08-14	10/6/2025/r
4	5	11004	AW00011004	Elizabeth	Johnson	Australia	Female	Single	1979-08-05	10/6/2025/r

Next steps: [View recommended plots](#) [New interactive sheet](#)

```

print(fact_sales.columns)
print(dim_customers.columns)
print(dim_products.columns)

```

```

Index(['order_number', 'customer_key', 'product_key', 'order_date',
       'shipping_date', 'due_date', 'sales', 'quantity', 'price'],
      dtype='object')
Index(['customer_key', 'customer_id', 'customer_number', 'first_name',
       'last_name', 'country', 'gender', 'marital_status', 'birth_date',
       'create_date'],
      dtype='object')
Index(['product_key', 'product_id', 'product_number', 'product_name',
       'category_id', 'category', 'sub_category', 'maintenance', 'product_line',
       'cost', 'start_date'],
      dtype='object')

```

➤ Merging Datasets:

```

[ ] sales_customers = pd.merge(fact_sales, dim_customers, on='customer_key', how='left')

[ ] full_data = pd.merge(sales_customers, dim_products, on='product_key', how='left')

```

➤ Data overview code:

```

[ ] full_data.head()

```

	order_number	customer_key	product_key	order_date	shipping_date	due_date	sales	quantity	price	customer_id	...	product_id	product_number	product_name	category_id
0	SO43897	10789	20	2010-12-29	2011-01-05	2011-01-10	3578	1	3578	21788	...	310	BK-R83R-82	Road-150 Red- 82	BI_RB
1	SO43898	17390	9	2010-12-29	2011-01-05	2011-01-10	3400	1	3400	28389	...	348	BK-M82S-44	Mountain-100 Silver- 44	BI_MB
2	SO43899	14884	9	2010-12-29	2011-01-05	2011-01-10	3400	1	3400	25883	...	348	BK-M82S-44	Mountain-100 Silver- 44	BI_MB
3	SO43700	3502	41	2010-12-29	2011-01-05	2011-01-10	899	1	899	14501	...	337	BK-R50B-82	Road-850 Black- 82	BI_RB
4	SO43701	4	9	2010-12-29	2011-01-05	2011-01-10	3400	1	3400	11003	...	348	BK-M82S-44	Mountain-100 Silver- 44	BI_MB

5 rows X 28 columns

```

[ ] full_data.shape

```

```

(68398, 28)

```

```
[ ] full_data.info()
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 60398 entries, 0 to 60397
Data columns (total 28 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_number          60398 non-null  object
1   customer_key          60398 non-null  int64
2   product_key           60398 non-null  int64
3   order_date            60379 non-null  object
4   shipping_date         60398 non-null  object
5   due_date              60398 non-null  object
6   sales                 60398 non-null  int64
7   quantity              60398 non-null  int64
8   price                 60398 non-null  int64
9   customer_id           60398 non-null  int64
10  customer_number       60398 non-null  object
11  first_name            60392 non-null  object
12  last_name             60392 non-null  object
13  country               59527 non-null  object
14  gender                60365 non-null  object
15  marital_status        60394 non-null  object
16  birth_date            60320 non-null  object
17  create_date           60398 non-null  object
18  product_id            60398 non-null  int64
19  product_number        60398 non-null  object
20  product_name          60398 non-null  object
21  category_id           60398 non-null  object
22  category              60398 non-null  object
23  sub_category          60398 non-null  object
24  maintenance           60398 non-null  object
25  product_line          60398 non-null  object
26  cost                  60398 non-null  int64
27  start_date            60398 non-null  object
dtypes: int64(8), object(20)
memory usage: 12.9+ MB
```

```
[ ] full_data.describe()
```

```
>>>
```

	customer_key	product_key	sales	quantity	price	customer_id	product_id	cost
count	60398.000000	60398.000000	60398.000000	60398.000000	60398.000000	60398.000000	60398.000000	60398.000000
mean	7842.685420	212.284331	488.042336	1.000414	488.058545	18841.685420	437.598480	292.563495
std	5432.430404	80.073598	928.452681	0.044011	928.452690	5432.430404	118.063313	563.249698
min	1.000000	3.000000	0.000000	1.000000	2.000000	11000.000000	214.000000	1.000000
25%	3004.000000	138.000000	8.000000	1.000000	8.000000	14003.000000	359.000000	3.000000
50%	7144.000000	248.000000	30.000000	1.000000	30.000000	18143.000000	479.000000	11.000000
75%	12430.750000	285.000000	540.000000	1.000000	540.000000	23429.750000	529.000000	295.000000
max	18484.000000	295.000000	3578.000000	10.000000	3578.000000	29483.000000	606.000000	2171.000000

➤ Initial Data Cleaning:

```
[ ] missing_counts = full_data.isnull().sum()
print("Missing values per column:\n", missing_counts[missing_counts > 0])
```

```
>>> Missing values per column:
order_date      19
first_name       6
last_name       6
country        871
gender          33
marital_status   4
birth_date      78
dtype: int64
```

```
[ ] for col in full_data.select_dtypes(include='object').columns:
    full_data[col] = full_data[col].str.strip()
```

```
[ ] # Example date columns, replace with your actual columns
date_cols = ['order_date', 'ship_date']

for col in date_cols:
    if col in full_data.columns:
        full_data[col] = pd.to_datetime(full_data[col], errors='coerce')
```

```

for col in full_data.select_dtypes(include='object').columns:
    print(f"Column: {col}")
    print(full_data[col].value_counts().head(10))

```

```

Column: order_number
order_number
S072656      8
S070714      8
S058845      8
S060233      7
S074869      7
S064542      7
S051555      7
S071961      7
S072927      7
S061412      7
Name: count, dtype: int64
Column: shipping_date
shipping_date
2013-12-19    263
2013-10-08    246
2013-06-12    241
2013-12-16    238
2013-08-26    237
2013-12-12    227
2013-12-22    222
2013-11-05    220
2014-01-02    220
2013-12-26    219
Name: count, dtype: int64
Column: due_date
due_date
2013-12-24    263
2013-10-13    246
2013-06-17    241
2013-12-21    238

```

```

[ ] import pandas as pd
import matplotlib.pyplot as plt

```

```

print("Columns in full_data:\n", full_data.columns.tolist())

```

```

Columns in full_data:
['order_number', 'customer_key', 'product_key', 'order_date', 'shipping_date', 'due_date', 'sales', 'quantity', 'price', 'cu

```

```

[ ] #Identify numeric columns
expected_numeric = ['sales', 'quantity', 'price']
numeric_cols = [c for c in expected_numeric if c in full_data.columns]
print("\nNumeric columns detected:", numeric_cols)

```

```

Numeric columns detected: ['sales', 'quantity', 'price']

```

```

[ ] #Coerce to numeric
for col in numeric_cols:
    full_data[col] = pd.to_numeric(full_data[col], errors='coerce')

```

```

#Missing values summary (counts + percent)
missing_counts = full_data.isnull().sum().sort_values(ascending=False)
missing_percent = (full_data.isnull().mean() * 100).round(3).sort_values(ascending=False)

print("\nMissing values (top):\n", missing_counts[missing_counts > 0].head(20))
print("\nMissing percent (top):\n", missing_percent[missing_percent > 0].head(20))

```

```

Missing values (top):
country      871
birth_date   78
gender       33
order_date   19
last_name    6
first_name   6
marital_status 4
dtype: int64

Missing percent (top):
country      1.442
birth_date   0.129
gender       0.055
order_date   0.031
last_name    0.010
first_name   0.010
marital_status 0.007
dtype: float64

```

```
#check if customer/product dimension info is missing after merge
if 'first_name' in full_data.columns:
    print(f"\nRows missing customer info (first_name): {full_data['first_name'].isnull().sum()}")
if 'product_name' in full_data.columns:
    print(f"\nRows missing product info (product_name): {full_data['product_name'].isnull().sum()}")
```

```
↵
Rows missing customer info (first_name): 6
Rows missing product info (product_name): 0
```

```
[ ] #Zero / negative value checks for numeric columns
n_rows = len(full_data)
for col in numeric_cols:
    zeros = (full_data[col] == 0).sum()
    negs = (full_data[col] < 0).sum()
    print(f"\nColumn '{col}': zeros = {zeros} ({zeros/n_rows:.4%}), negatives = {negs} ({negs/n_rows:.4%})")
```

```
↵
Column 'sales': zeros = 7 (0.0116%), negatives = 0 (0.0000%)
Column 'quantity': zeros = 0 (0.0000%), negatives = 0 (0.0000%)
Column 'price': zeros = 0 (0.0000%), negatives = 0 (0.0000%)
```

```
▶ if 'quantity' in numeric_cols:
    q_invalid = full_data[full_data['quantity'] <= 0].shape[0]
    print(f"\nRows with quantity <= 0: {q_invalid}")

    if 'price' in numeric_cols:
        p_invalid = full_data[full_data['price'] <= 0].shape[0]
        print(f"Rows with price <= 0: {p_invalid}")
```

```
↵
Rows with quantity <= 0: 0
Rows with price <= 0: 0
```

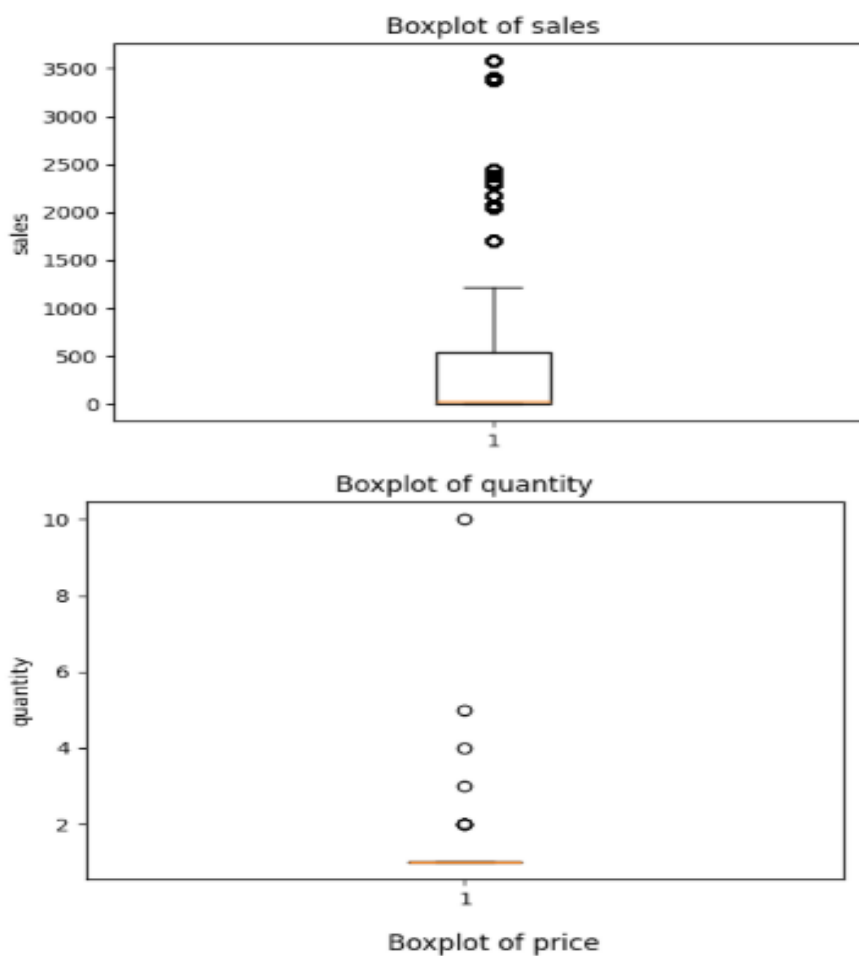
```
▶ #Outlier detection using IQR for each numeric column
outlier_summary = {}
for col in numeric_cols:
    col_ser = full_data[col].dropna()
    Q1 = col_ser.quantile(0.25)
    Q3 = col_ser.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers_mask = (full_data[col] < lower) | (full_data[col] > upper)
    outlier_count = outliers_mask.sum()
    outlier_summary[col] = {
        'Q1': Q1, 'Q3': Q3, 'IQR': IQR,
        'lower_bound': lower, 'upper_bound': upper,
        'outlier_count': int(outlier_count),
        'outlier_pct': float(outlier_count / n_rows * 100)
    }
    print(f"\nOutliers in '{col}': {outlier_count} rows ({outlier_count/n_rows:.4%})")
# show a few extreme rows for inspection
print(full_data.loc[outliers_mask, [col, 'order_number', 'customer_key', 'product_key']].head(5))
```

```
↵
Outliers in 'sales': 9586 rows (15.8714%)
  sales order_number customer_key product_key
0   3578      5043697      18769          20
1   3400      5043698      17390           9
2   3400      5043699      14864           9
4   3400      5043701           4           9
5   3578      5043702      16646          16

Outliers in 'quantity': 11 rows (0.0182%)
  quantity order_number customer_key product_key
5658         2      5051259          434         295
5778         5      5051298      16950         295
6018         2      5051387           943         245
6997        10      5051942          224         104
7567         2      5052187       7111         168

Outliers in 'price': 9586 rows (15.8714%)
  price order_number customer_key product_key
0   3578      5043697      18769          20
1   3400      5043698      17390           9
2   3400      5043699      14864           9
4   3400      5043701           4           9
5   3578      5043702      16646          16
```

```
#Visual boxplots (one chart per numeric column)
for col in numeric_cols:
    plt.figure(figsize=(6,4))
    plt.title(f"Boxplot of {col}")
    # remove NaNs for plotting
    plt.boxplot(full_data[col].dropna())
    plt.ylabel(col)
    plt.show()
```



```
[ ] #Handle missing values
full_data['country'] = full_data['country'].fillna('Unknown')
full_data['birth_date'] = full_data['birth_date'].fillna(pd.NaT)
full_data['gender'] = full_data['gender'].fillna('Unknown')
```

```
[ ] zero_sales_count = int((full_data['sales'] == 0).sum()) if 'sales' in full_data.columns else 0
total_rows = len(full_data)
print(f"\nRows with sales == 0: {zero_sales_count} / {total_rows} ({zero_sales_count/total_rows:.4%})")
```



Rows with sales == 0: 7 / 60398 (0.0116%)

```
[ ] full_data_clean = full_data.copy()
```

```
[ ] #drop rows with sales <= 0 for revenue calculations / queries
if 'sales' in full_data_clean.columns:
    full_data_revenue = full_data_clean[full_data_clean['sales'] > 0].copy()
else:
    full_data_revenue = full_data_clean.copy()
```

```
[ ] print("\nShapes:")
print(" full_data      :", full_data.shape)
print(" full_data_clean :", full_data_clean.shape)
print(" full_data_revenue (sales>0) :", full_data_revenue.shape)
```



Shapes:

```
full_data      : (60398, 28)
full_data_clean : (60398, 28)
full_data_revenue (sales>0) : (60391, 28)
```

```
[ ] full_data_revenue[['sales', 'quantity', 'price']].describe()
```



	sales	quantity	price
count	60391.000000	60391.000000	60391.000000
mean	486.098674	1.000149	486.092348
std	928.491742	0.016777	928.494294
min	2.000000	1.000000	2.000000
25%	8.000000	1.000000	8.000000
50%	30.000000	1.000000	30.000000
75%	540.000000	1.000000	540.000000
max	3578.000000	4.000000	3578.000000

```
[ ] date_cols = ['order_date', 'shipping_date', 'due_date', 'birth_date', 'create_date', 'start_date']
for col in date_cols:
    full_data_revenue[col] = pd.to_datetime(full_data_revenue[col], errors='coerce')
```

```
[ ] full_data_revenue = full_data_revenue.drop_duplicates()
```

➤ EDA of cleaned dataset:

```
[ ] #EDA Process Starts here
df = full_data_revenue.copy()
```

```
[ ] df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
Index: 60391 entries, 0 to 60397
Data columns (total 28 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_number          60391 non-null  object
1   customer_key          60391 non-null  int64
2   product_key           60391 non-null  int64
3   order_date            60372 non-null  datetime64[ns]
4   shipping_date         60391 non-null  datetime64[ns]
5   due_date              60391 non-null  datetime64[ns]
6   sales                 60391 non-null  int64
7   quantity              60391 non-null  int64
8   price                 60391 non-null  int64
9   customer_id           60391 non-null  int64
10  customer_number       60391 non-null  object
11  first_name            60385 non-null  object
12  last_name             60385 non-null  object
13  country               60391 non-null  object
14  gender                60391 non-null  object
15  marital_status        60387 non-null  object
16  birth_date            60313 non-null  datetime64[ns]
17  create_date           60391 non-null  datetime64[ns]
18  product_id            60391 non-null  int64
19  product_number        60391 non-null  object
20  product_name          60391 non-null  object
21  category_id           60391 non-null  object
22  category              60391 non-null  object
23  sub_category          60391 non-null  object
24  maintenance           60391 non-null  object
25  product_line          60391 non-null  object
26  cost                  60391 non-null  int64
27  start_date            60391 non-null  datetime64[ns]
dtypes: datetime64[ns](6), int64(8), object(14)
memory usage: 13.4+ MB
```

```
[ ] df.describe()
```

	customer_key	product_key	order_date	shipping_date	due_date	sales	quantity	price	customer_id	birth_date	create_date
count	80391.000000	80391.000000	80372	80391	80391	80391.000000	80391.000000	80391.000000	80391.000000	80313	80391
mean	7842.981173	212.284827	2013-08-09 12:10:18.815742208	2013-08-18 12:45:50.478084256	2013-08-21 12:45:50.478084256	486.098874	1.000149	486.092348	18841.981173	1989-05-28 00:40:10.942914464	2025-12-09 01:57:31.805732608
min	1.000000	3.000000	2010-12-29 00:00:00	2011-01-05 00:00:00	2011-01-10 00:00:00	2.000000	1.000000	2.000000	11000.000000	1916-02-10 00:00:00	2025-10-08 00:00:00
25%	3004.000000	138.000000	2013-04-03 00:00:00	2013-04-10 00:00:00	2013-04-15 00:00:00	8.000000	1.000000	8.000000	14003.000000	1981-12-08 00:00:00	2025-10-12 00:00:00
50%	7145.000000	246.000000	2013-07-14 00:00:00	2013-07-21 00:00:00	2013-07-28 00:00:00	30.000000	1.000000	30.000000	18144.000000	1971-08-15 00:00:00	2026-01-04 00:00:00
75%	12431.500000	285.000000	2013-10-18 00:00:00	2013-10-25 00:00:00	2013-10-30 00:00:00	540.000000	1.000000	540.000000	23430.500000	1978-07-15 00:00:00	2026-01-05 00:00:00
max	18484.000000	295.000000	2014-01-28 00:00:00	2014-02-04 00:00:00	2014-02-09 00:00:00	3578.000000	4.000000	3578.000000	29483.000000	1988-08-25 00:00:00	2026-01-25 00:00:00
std	5432.359185	80.074003	NaN	NaN	NaN	928.491742	0.016777	928.494294	5432.359185	NaN	NaN

```
[ ] df.isnull().sum()
```

	e
order_number	0
customer_key	0
product_key	0
order_date	19
shipping_date	0
due_date	0
sales	0
quantity	0
price	0
customer_id	0
customer_number	0
first_name	6
last_name	6
country	0
gender	0
marital_status	4
birth_date	78
create_date	0
product_id	0
product_number	0
product_name	0
category_id	0
category	0
sub_category	0
maintenance	0
product_line	0
cost	0
start_date	0

dtype: int64

➤ Each major query or group of queries:



```
# Revenue Analysis
# 1. Total revenue
df['sales'].sum()
```



```
np.int64(29355985)
```

```
[ ] # 2. Total revenue by year
df.groupby(df['order_date'].dt.year)['sales'].sum()
```

```
↕
sales
order_date
2010.0    43419
2011.0   7075088
2012.0   5842231
2013.0  16344613
2014.0    45642
dtype: int64
```

```
[ ] # 3. Average sales per order
df['sales'].mean()
```

```
↕ np.float64(486.0986736434237)
```

```
[ ] # 4. Highest single order value
df['sales'].max()
```

```
↕ 3578
```

```
[ ] # 5. Lowest non-zero order value
df[df['sales'] > 0]['sales'].min()
```

```
↕ 2
```

```
[ ] # 6. Revenue contribution % by each country
(df.groupby('country')['sales'].sum() / df['sales'].sum()) * 100
```

```
↕
sales
country
Australia    30.862729
Canada       6.736746
France       9.005799
Germany      9.858521
United Kingdom 11.552503
United States 31.211049
Unknown      0.772653
dtype: float64
```

```
[ ] # 7. Top 10 revenue-generating customers
df.groupby('customer_key')['sales'].sum().nlargest(10)
```

```
↕
sales
customer_key
1133    13294
1302    13294
1309    13268
1132    13265
1301    13242
1322    13215
1125    13195
1308    13172
1297    13164
434     12904
dtype: int64
```

```
[ ] # 8. Total number of unique customers
df['customer_key'].unique()
```

```
↕ 18484
```

```
[ ] # Product Analysis
# 9. Top 5 selling products by total revenue
df.groupby('product_name')['sales'].sum().nlargest(5)
```

```
↕
sales
product_name
Mountain-200 Black- 46  1373454
Mountain-200 Black- 42  1363128
Mountain-200 Silver- 38 1339394
Mountain-200 Silver- 46 1301029
Mountain-200 Black- 38  1294854
dtype: int64
```

```
[ ] # 10. Top 5 selling products by quantity sold
df.groupby('product_name')['quantity'].sum().nlargest(5)
```

```
↕
quantity
product_name
Water Bottle - 30 oz.    4242
Patch Kit/8 Patches     3191
Mountain Tire Tube      3098
Road Tire Tube          2376
Sport-100 Helmet- Red   2230
dtype: int64
```

```
[ ] # 11. Products with zero quantity sold (if any)
df[df['quantity'] == 0]['product_name'].unique()
```

```
↕ array([], dtype=object)
```

```
[ ] # 12. Average selling price (ASP) per product
(df.groupby('product_name')['sales'].sum() / df.groupby('product_name')['quantity'].sum()).sort_values(ascending=False)
```

```
0
product_name
Road-150 Red- 56  3578.0
Road-150 Red- 62  3578.0
Road-150 Red- 48  3578.0
Road-150 Red- 44  3578.0
Road-150 Red- 52  3578.0
...
Mountain Tire Tube  5.0
Touring Tire Tube  5.0
Water Bottle - 30 oz.  5.0
Road Tire Tube  4.0
Patch Kit/8 Patches  2.0
130 rows x 1 columns
dtype: float64
```

```
[ ] # 13. Most frequently purchased product
df['product_name'].mode()[0]
```

```
'Water Bottle - 30 oz.'
```

```
[ ] # Customer Insights
# 14. Average revenue per customer
df.groupby('customer_key')['sales'].mean().sort_values(ascending=False)
```

```
sales
customer_key
16578  3578.0
16579  3578.0
16602  3578.0
16605  3578.0
16606  3578.0
...
17777  2.0
17096  2.0
17782  2.0
16994  2.0
17301  2.0
18484 rows x 1 columns
dtype: float64
```

```
[ ] # 15. Customers who placed the most orders
df.groupby('customer_key')['order_number'].nunique().sort_values(ascending=False).head(10)
```

```
order_number
customer_key
92  28
177  28
201  27
186  27
712  27
278  27
332  27
331  27
224  27
301  27
dtype: int64
```

```
# 16. Revenue by marital status
df.groupby('marital_status')['sales'].sum()
```

```
sales
marital_status
Married  15180485
Single  14187677
dtype: int64
```

```
[ ] # 17. Revenue by gender
df.groupby('gender')['sales'].sum()
```

```
sales
gender
Female  14804068
Male  14522228
Unknown  29689
dtype: int64
```

```
# 18. Top 5 customers by average order value
(df.groupby('customer_key')['sales'].sum() / df.groupby('customer_key')['order_number'].nunique()).nlargest(5)
```

```
0
customer_key
11972      3578.0
11976      3578.0
12567      3578.0
12807      3578.0
13492      3578.0

dtype: float64
```

```
[ ] # Time-based Trends
# 19. Monthly sales trend
df.groupby(df['order_date'].dt.to_period('M'))['sales'].sum()
```

```
sales
order_date
2010-12      43419
2011-01     469795
2011-02     466307
2011-03     485165
2011-04     502042
2011-05     561647
2011-06     737793
2011-07     596710
```

```
# 20. Day-of-week sales trend
df.groupby(df['order_date'].dt.day_name())['sales'].sum()
```

```
sales
order_date
Friday      4152865
Monday      4234993
Saturday     4124380
Sunday      4113316
Thursday     4152051
Tuesday     4342180
Wednesday   4231208

dtype: int64
```

```
[ ] # 21. Yearly sales growth
df.groupby(df['order_date'].dt.year)['sales'].sum().pct_change() * 100
```

```
sales
order_date
2010.0      NaN
2011.0     16194.912366
2012.0     -17.425324
2013.0     179.766634
2014.0    -99.720752

dtype: float64
```

```
[ ] # 22. Highest revenue month for each year
df.groupby([df['order_date'].dt.year, df['order_date'].dt.month])['sales'].sum().groupby(level=0).idxmax()
```

```

sales

order_date
2010.0    (2010.0, 12.0)
2011.0    (2011.0, 8.0)
2012.0    (2012.0, 12.0)
2013.0    (2013.0, 12.0)
2014.0    (2014.0, 1.0)

```

dtype: object

```
[ ] # 23. Monthly revenue trend
df['order_date'] = pd.to_datetime(df['order_date'])
monthly_revenue = df.groupby(df['order_date'].dt.to_period('M'))['sales'].sum().reset_index()
monthly_revenue['order_date'] = monthly_revenue['order_date'].astype(str)
monthly_revenue
```

```

order_date  sales
0    2010-12   43419
1    2011-01  488795
2    2011-02  488307
3    2011-03  485165
4    2011-04  502042
5    2011-05  581847

```

```
[ ] # 24. Yearly total sales
yearly_sales = df.groupby(df['order_date'].dt.year)['sales'].sum().reset_index(name='YearlyRevenue')
yearly_sales
```

```

order_date  YearlyRevenue
0    2010.0           43419
1    2011.0       7075088
2    2012.0       5842231
3    2013.0      18344813
4    2014.0        45842

```

```
[ ] # 25. Weekday vs Weekend sales comparison
df['DayOfWeek'] = df['order_date'].dt.day_name()
weekday_vs_weekend = df.groupby(
    df['DayOfWeek'].isin(['Saturday', 'Sunday'])
)['sales'].sum().reset_index(name='Revenue')
weekday_vs_weekend['Type'] = ['Weekday', 'Weekend']
weekday_vs_weekend
```

```

DayOfWeek  Revenue  Type
0    False  21118289  Weekday
1     True   8237896  Weekend

```

```
[ ] # Regional / Categorical Analysis
# 26. Sales by country
df.groupby('country')['sales'].sum().sort_values(ascending=False).head(6)
```

↕

sales	
country	
United States	9162311
Australia	9060058
United Kingdom	3391351
Germany	2894066
France	2643741
Canada	1977638

dtype: int64

```
[ ] # 27. Sales by product category
df.groupby('category')['sales'].sum().sort_values(ascending=False).head(10)
```

↕

sales	
category	
Bikes	28316272
Accessories	6999997
Clothing	339716

dtype: int64

```
[ ] # 28. Top selling product in each country
df.groupby(['country', 'product_name'])['sales'].sum() \
.reset_index() \
.sort_values(['country', 'sales'], ascending=[True, False]) \
.groupby('country').head(1)
```

↕

	country	product_name	sales
81	Australia	Road-150 Red- 62	397158
182	Canada	Mountain-200 Black- 42	131715
281	France	Mountain-200 Black- 46	138600
424	Germany	Mountain-200 Silver- 46	163803
560	United Kingdom	Mountain-200 Black- 42	183516
831	United States	Mountain-200 Black- 46	525057
811	Unknown	Road-550-W Yellow- 40	30240

🔊 # Advanced / RFM(Recency, Frequency, Monetary)-style Metrics
29. Recency: Days since last purchase per customer
(df.groupby('customer_key')['order_date'].max() - df['order_date'].max()).dt.days

↕

order_date	
customer_key	
1	-270.0
2	-49.0
3	-339.0
4	-263.0
6	-272.0
...	...
18480	-511.0
18481	-195.0
18482	-899.0
18483	-497.0
18484	-506.0

18484 rows × 1 columns

dtype: float64

```
# 30. Frequency: Total number of orders per customer
df.groupby('customer_key')['order_number'].nunique()
```

order_number	
customer_key	
1	3
2	3
3	3
4	3
5	3
...	...
18480	1
18481	1
18482	1
18483	1
18484	1

18484 rows × 1 columns

dtype: int64

```
# 31. Monetary: Total spend per customer
df.groupby('customer_key')['sales'].sum()
```

sales	
customer_key	
1	8249
2	6384
3	8114
4	8139
5	8196
...	...
18480	2049
18481	2442
18482	3375
18483	2049
18484	2049

18484 rows × 1 columns

dtype: int64

```
# 32. Revenue from repeat customers (more than 1 order)
df[df['customer_key'].isin(df['customer_key'].value_counts()[df['customer_key'].value_counts() > 1].index)][['sales']].sum()
```

np.int64(27420374)

```
# 33. Customers who spent above overall average revenue
df.groupby('customer_key')['sales'].sum()[df.groupby('customer_key')['sales'].sum() > df['sales'].mean()]
```

sales	
customer_key	
1	8249
2	6384
3	8114
4	8139
5	8196
...	...
18480	2049
18481	2442
18482	3375
18483	2049
18484	2049

9169 rows × 1 columns

dtype: int64

```
# 34. Recency: Days since last purchase per customer
latest_date = df['order_date'].max()
recency_df = df.groupby('customer_key')['order_date'].max().reset_index()
recency_df['RecencyDays'] = (latest_date - recency_df['order_date']).dt.days
recency_df
```

	customer_key	order_date	RecencyDays
0	1	2013-05-03	270.0
1	2	2013-12-10	49.0
2	3	2013-02-23	339.0
3	4	2013-05-10	263.0
4	5	2013-05-01	272.0
...	...	...	...
18479	18480	2012-09-04	511.0
18480	18481	2013-07-17	195.0
18481	18482	2011-08-13	899.0
18482	18483	2012-09-18	497.0
18483	18484	2012-09-09	506.0

18484 rows × 3 columns

```
[ ] # 35. Frequency: Number of unique orders per customer
frequency_df = df.groupby('customer_key')['order_number'].nunique().reset_index(name='Frequency')
frequency_df
```

	customer_key	Frequency
0	1	3
1	2	3
2	3	3
3	4	3
4	5	3
...	...	...
18479	18480	1
18480	18481	1
18481	18482	1
18482	18483	1
18483	18484	1

18484 rows x 2 columns

```
[ ] # 36. Monetary: Total spend per customer
monetary_df = df.groupby('customer_key')['sales'].sum().reset_index(name='MonetaryValue')
monetary_df
```

	customer_key	MonetaryValue
0	1	8249
1	2	6384
2	3	8114
3	4	8139
4	5	8198
...	...	...
18479	18480	2049
18480	18481	2442
18481	18482	3375
18482	18483	2049
18483	18484	2049

18484 rows x 2 columns

```
[ ] # Correlation & Stats
# 37. Correlation between quantity and sales
correlation_qty_sales = df[['quantity', 'sales']].corr().iloc[0, 1]
correlation_qty_sales
```

 np.float64(-0.00337702305069065)



38. Top 5% customers by revenue contribution

```
threshold = df.groupby('customer_key')['sales'].sum().quantile(0.95)
```

```
top_5_percent_customers = df.groupby('customer_key')['sales'].sum().reset_index()
```

```
top_5_percent_customers = top_5_percent_customers[top_5_percent_customers['sales'] >= threshold]
```

```
top_5_percent_customers
```



	customer_key	sales
0	1	8249
1	2	6384
2	3	8114
3	4	8139
4	5	8196
...	...	...
17243	17244	6061
17244	17245	6026
17255	17256	6051
17259	17260	6118
17288	17289	6039

959 rows × 2 columns

```
[ ] # Category & Trend Insights
# 39. Most popular product each month (by quantity sold)
df['Month'] = df['order_date'].dt.month
most_popular_per_month = df.groupby(['Month', 'product_name'])['quantity'].sum().reset_index()
most_popular_per_month = most_popular_per_month.sort_values(['Month', 'quantity'], ascending=[True, False])
most_popular_per_month.groupby('Month').first().reset_index()
```



	Month	product_name	quantity
0	1.0	Water Bottle - 30 oz.	274
1	2.0	Water Bottle - 30 oz.	274
2	3.0	Water Bottle - 30 oz.	319
3	4.0	Water Bottle - 30 oz.	307
4	5.0	Water Bottle - 30 oz.	312
5	6.0	Water Bottle - 30 oz.	395
6	7.0	Water Bottle - 30 oz.	355
7	8.0	Water Bottle - 30 oz.	405
8	9.0	Water Bottle - 30 oz.	376
9	10.0	Water Bottle - 30 oz.	408
10	11.0	Water Bottle - 30 oz.	384
11	12.0	Water Bottle - 30 oz.	433



```
# 40. Cumulative revenue over time
df_sorted = df.sort_values('order_date')
df_sorted['CumulativeRevenue'] = df_sorted['sales'].cumsum()
df_sorted[['order_date', 'CumulativeRevenue']].head(21)
```



	order_date	CumulativeRevenue
0	2010-12-29	3578
1	2010-12-29	6978
2	2010-12-29	10378
3	2010-12-29	11077
4	2010-12-29	14477
5	2010-12-30	18055
6	2010-12-30	21633
7	2010-12-30	25008
8	2010-12-30	28408
9	2010-12-31	31986
10	2010-12-31	35564
11	2010-12-31	36263
12	2010-12-31	39841
13	2010-12-31	43419
15	2011-01-01	46997
14	2011-01-01	50575
16	2011-01-02	54153
17	2011-01-02	57731
18	2011-01-02	61309
19	2011-01-02	64887
20	2011-01-02	65586

❖ **Key Analysis Performed:**

- Sales Trends: Monthly, yearly and seasonal analysis.
- Product Performance: Best and worst-selling items, revenue by category & sub-category.
- Customer Insights:
 - Recency – Days since last purchase.
 - Frequency – Number of orders per customer.
 - Monetary – Total spend per customer.
- Geographic Performance: Country-wise sales & profit analysis.
- Discount Impact: Relationship between discounts and profitability.

❖ **Total Queries Executed:**

- **40** unique Pandas queries, all written and executed in Python for EDA.

❖ Outcome:

- The analysis revealed:
 - Top revenue-generating products and categories.
 - Most valuable customers based on purchase frequency and spend.
 - Seasonal peaks in sales performance.
 - Countries contributing most to revenue.
 - Insights into the negative impact of high discounts on profit.

❖ Conclusion:

- This project showcases how **Python and Pandas** can be leveraged for exploratory data analysis in retail. By breaking down sales and customer data into meaningful patterns, businesses can identify growth opportunities, optimize inventory and improve customer targeting strategies.

❖ Appendix / References:

- For full code and execution details:
 - [Click here to view my Colab Notebook](#)